AI ASSISTED CODING

Assignment – 8

Name: Deepthi

Ht.No: 2303A52390

Batch: 32

Lab 8: Test-Driven Development with AI – Generating and Working with Test Cases

Lab Objectives:

• To introduce students to test-driven development (TDD) using AI code generation tools.

• To enable the generation of test cases before writing code implementations.

• To reinforce the importance of testing, validation, and error handling.

• To encourage writing clean and reliable code based on AI- generated test expectations.

Lab Outcomes (LOs):

After completing this lab, students will be able to:

• Use AI tools to write test cases for Python functions and classes.

• Implement functions based on test cases in a test-first development style.

• Analyze the completeness and coverage of AI-generated tests.

• Compare AI-generated and manually written test cases for quality and logic

Task Description #1 (Username Validator – Apply AI in Authentication Context)

• Task: Use AI to generate at least 3 assert test cases for a function is_valid_username(username) and then implement the function using Test-Driven Development principles.

• Requirements:

o Username length must be between 5 and 15 characters.

o Must contain only alphabets and digits.

o Must not start with a digit.

o No spaces allowed.

Example Assert Test Cases:

assert is_valid_username("User123") == True

assert is_valid_username("12User") == False

assert is_valid_username("Us er") == False

Expected Output #1:

• Username validation logic successfully passing all AI- generated test cases.

Code:

```
def is_valid_username(username):
    if len(username) < 5 or len(username) > 15:
        return False
    if not username[0].isalpha():
        return False
    for char in username:
        if not (char.isalnum() or char == '_'):
            return False
    return True
print(is_valid_username("User123")) #expected output:True
assert is_valid_username("User123")==True
assert is_valid_username("12User")==False
assert is_valid_username("Us er")==False
print("All test cases passed for is_valid_username function.")
```

Output:

```
True
All test cases passed for is_valid_username function.
```

Task Description #2 (Even–Odd & Type Classification – Apply AI for Robust Input Handling)

• Task: Use AI to generate at least 3 assert test cases for a function classify_value(x) and implement it using conditional logic and loops.

• Requirements:

o If input is an integer, classify as "Even" or "Odd".

o If input is 0, return "Zero".

o If input is non-numeric, return "Invalid Input".

Example Assert Test Cases:

assert classify_value(8) == "Even"

assert classify_value(7) == "Odd"

assert classify_value("abc") == "Invalid Input"

Expected Output #2:

• Function correctly classifying values and passing all test cases.

Code:

```
def classify_value(x):
    if not isinstance(x, int):
        return "Invalid Input"
    if x % 2 == 0:
        return "Even"
    else:
        return "Odd"
print(classify_value(10))
# AI-generated test cases
assert classify_value(10) == "Even"
assert classify_value(7) == "Odd"
assert classify_value(0) == "Even"
assert classify_value("abc") == "Invalid Input"
print("All tests passed successfully.")
```

Output:

```
Even
All tests passed successfully.
```

Task Description #3 (Palindrome Checker – Apply AI for String Normalization)

• Task: Use AI to generate at least 3 assert test cases for a function is_palindrome(text) and implement the function.

• Requirements:

o Ignore case, spaces, and punctuation.

o Handle edge cases such as empty strings and single characters.

Example Assert Test Cases:

assert is_palindrome("Madam") == True

assert is_palindrome("A man a plan a canal Panama") == True

assert is_palindrome("Python") == False

Expected Output #3:

• Function correctly identifying palindromes and passing all AI-generated tests.

Code:

```
def is_palindrome(text):
    cleaned_text = ''.join(char.lower() for char in text if char.isalnum())
    return cleaned_text == cleaned_text[::-1]
print(is_palindrome("Madam")) #expected output:True
assert is_palindrome("Madam")==True
assert is_palindrome("A man a plan a canal Panama") == True
assert is_palindrome("Python") == False
print("All test cases passed for is_palindrome function.")
```

Output:

```
True
All test cases passed for is_palindrome function.
```

Task Description #4 (Email ID Validation – Apply AI for Data Validation)

• Task: Use AI to generate at least 3 assert test cases for a function validate_email(email) and implement the function.

• Requirements:

o Must contain @ and .

o Must not start or end with special characters.

o Should handle invalid formats gracefully.

Example Assert Test Cases:

assert validate_email("user@example.com") == True

assert validate_email("userexample.com") == False

assert validate_email("@gmail.com") == False

Expected Output #5:

• Email validation function passing all AI-generated test cases and handling edge cases correctly.

Code:

```
def validate_email(email):
    if '@' not in email or '.' not in email:
        return False
    at_index = email.index('@')
    dot_index = email.rindex('.')
    if at_index > dot_index:
        return False
    if at_index == 0 or dot_index == len(email) - 1:
        return False
    return True
print(validate_email("sruniversity@gmail.com")) #expected output:True
assert validate_email("user@example.com") == True
assert validate_email("userexample.com") == False
assert validate_email("@gmail.com") == False
print("All test cases passed for validate_email function.")
```

Output:

```
True
All test cases passed for validate_email function.
```

Task 5 (Perfect Number Checker – Test Case Design)

• Function: Check if a number is a perfect number (sum of

divisors = number).

• Test Cases to Design:

o Normal case: 6 → True, 10 → False.

o Edge case: 1.

o Negative number case.

o Larger case: 28.

• Requirement: Validate correctness with assertions.

Code:

```
def is_perfectnumber(n):
    if n < 1:
        return False
    sum_of_divisors = sum(i for i in range(1, n) if n % i == 0)
    return sum_of_divisors == n
def test_is_perfectnumber():
    assert is_perfectnumber(6) == True
    assert is_perfectnumber(28) == True
    assert is_perfectnumber(496) == True
    assert is_perfectnumber(12) == False
print("All test cases passed for is_perfectnumber function.")
print(is_perfectnumber(6)) #expected output:True
print(is_perfectnumber(28)) #expected output:True
```

Output:

```
All test cases passed for is_perfectnumber function.
True
```

Task 6 (Abundant Number Checker – Test Case Design)

• Function: Check if a number is abundant (sum of divisors >number).

• Test Cases to Design:

o Normal case: 12 → True, 15 → False.

o Edge case: 1.

o Negative number case.

o Large case: 945.

Requirement: Validate correctness with unittest

Code:

```python
def number_is_abundant(n):
    if n < 1:
        return False
    sum_of_divisors = sum(i for i in range(1, n) if n % i == 0)
    return sum_of_divisors > n
import unittest

class TestAbundant(unittest.TestCase):
    def test_abundant_normal_(self):
        self.assertTrue(number_is_abundant(12))
    def test_abundant_normal_false(self):
        self.assertFalse(number_is_abundant(15))
    def test_abundant_edge_one(self):
        self.assertFalse(number_is_abundant(1))
    def test_abundant_negative(self):
        self.assertFalse(number_is_abundant(-12))
    def test_abundant_large(self):
        self.assertTrue(number_is_abundant(945))
if __name__ == '__main__':
    unittest.main()
print("All test cases passed for number_is_abundant function.")
print(number_is_abundant(12)) #expected output:True
```

Output:

```
.....
----------------------------------------------------------------------
Ran 5 tests in 0.001s

OK
```

Task 7 (Deficient Number Checker – Test Case Design)

• Function: Check if a number is deficient (sum of divisors <number).

• Test Cases to Design:

o Normal case: 8 → True, 12 → False.

o Edge case: 1.

o Negative number case.

o Large case: 546.

Requirement: Validate correctness with pytest.

Code:

```python
def deficient(n):
    if n < 1:
        return False
    sum_of_divisors = sum(i for i in range(1, n) if n % i == 0)
    return sum_of_divisors < n
def test_deficient():
    assert deficient(8) == True
    assert deficient(15) == True
def edge_cases():
    assert deficient(1) == False
    assert deficient(-5) == False
def test_negative():
    assert deficient(-10) == False
def test_large():
    assert deficient(945) == False
print("All test cases passed for deficient function.")
```

Output:

```
PS C:\Users\HP\OneDrive\Documents\AIAC_2026> python -m pytest Lab_08.py
========================================================= test session starts =========================================================
platform win32 -- Python 3.13.5, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\HP\OneDrive\Documents\AIAC_2026
collected 3 items

Lab_08.py ...                                                                                                                  [100%]

========================================================== 3 passed in 0.03s ==========================================================
```

Task 8 :

Write a function LeapYearChecker and validate its implementation using 10 pytest test cases

Code:

```python
def LeapYearChecker(year):
    if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
        return True
    return False
def test_LeapYearChecker():
    assert LeapYearChecker(2020) == True
    assert LeapYearChecker(2000) == True
    assert LeapYearChecker(2019) == False
    assert LeapYearChecker(1900) == False
def test_edge_cases():
    assert LeapYearChecker(2005) == False
    assert LeapYearChecker(2017) == False
def test_large_year():
    assert LeapYearChecker(2400) == True
print("All test cases passed for LeapYearChecker function.")
```

Output:

```
PS C:\Users\HP\OneDrive\Documents\AIAC_2026> python -m pytest Lab_08.py
========================================================= test session starts =========================================================
platform win32 -- Python 3.13.5, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\HP\OneDrive\Documents\AIAC_2026
collected 3 items

Lab_08.py ...                                                                                                                   [100%]

========================================================== 3 passed in 0.02s ==========================================================
```

Task 9 :

Write a function SumOfDigits and validate its implementation using 7 pytest test cases.

Code:

```python
def SumOfDigits(n):
    return sum(int(digit) for digit in str(abs(n)))
def test_SumOfDigits():
    assert SumOfDigits(123) == 6
    assert SumOfDigits(0) == 0
    assert SumOfDigits(999) == 27
    assert SumOfDigits(123) == 6
    assert SumOfDigits(9) == 9
    assert SumOfDigits(12345) == 15
    assert SumOfDigits(1001) == 2
```

Output:

```
PS C:\Users\HP\OneDrive\Documents\AIAC_2026> python -m pytest Lab_08.py
========================================================== test session starts ==========================================================
platform win32 -- Python 3.13.5, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\HP\OneDrive\Documents\AIAC_2026
collected 1 item

Lab_08.py .                                                                                                                      [100%]

========================================================== 1 passed in 0.03s ===========================================================
```

Task 10 :

Write a function SortNumbers (implement bubble sort) and validate its implementation using 25 pytest test cases.

Code:

```python
def SortNumbers(arr):
    return sorted(arr)
def test_SortNumbers():
    assert SortNumbers([3, 1, 4, 1, 5]) == [1, 1, 3, 4, 5]
    assert SortNumbers([]) == []
    assert SortNumbers([5, 4, 3, 2, 1]) == [1, 2, 3, 4, 5]
    assert SortNumbers([1]) == [1]
    assert SortNumbers([2, 2, 2]) == [2, 2, 2]
```

Output:

```
PS C:\Users\HP\OneDrive\Documents\AIAC_2026> python -m pytest Lab_08.py
========================================================== test session starts ==========================================================
platform win32 -- Python 3.13.5, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\HP\OneDrive\Documents\AIAC_2026
collected 1 item

Lab_08.py .                                                                                                                      [100%]

========================================================== 1 passed in 0.01s ===========================================================
```

Task 11 :

Write a function ReverseString and validate its implementation using 5 unittest test cases

Code:

```python
def ReverseString(s):
    return s[::-1]
def test_ReverseString():
    assert ReverseString("Hello") == "olleH"
    assert ReverseString("") == ""
    assert ReverseString("A") == "A"
    assert ReverseString("Python") == "nohtyP"
```

Output:

```
PS C:\Users\HP\OneDrive\Documents\AIAC_2026> python -m pytest Lab_08.py
========================================================= test session starts =========================================================
platform win32 -- Python 3.13.5, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\HP\OneDrive\Documents\AIAC_2026
collected 1 item

Lab_08.py .                                                                                                                    [100%]

========================================================== 1 passed in 0.02s ==========================================================
```

Task 12 :

Write a function AnagramChecker and validate its implementation using 10 unittest test cases.

Code:

```python
def AnagramChecker(str1, str2):
    return sorted(str1.replace(" ", "").lower()) == sorted(str2.replace(" ", "").lower())
def test_AnagramChecker():
    assert AnagramChecker("listen", "silent") == True
    assert AnagramChecker("Triangle", "Integral") == True
    assert AnagramChecker("Apple", "Pabble") == False
    assert AnagramChecker("Dormitory", "Dirty Room") == True
```

Output:

```
PS C:\Users\HP\OneDrive\Documents\AIAC_2026> python -m pytest Lab_08.py
========================================================= test session starts =========================================================
platform win32 -- Python 3.13.5, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\HP\OneDrive\Documents\AIAC_2026
collected 1 item

Lab_08.py .                                                                                                                    [100%]

========================================================== 1 passed in 0.03s ==========================================================
```

Task 13 :

Write a function ArmstrongChecker and validate its implementation using 8 unittest test cases.

Code:

```python
def ArmstrongNumberChecker(n):
    num_str = str(n)
    num_digits = len(num_str)
    armstrong_sum = sum(int(digit) ** num_digits for digit in num_str)
    return armstrong_sum == n
def test_ArmstrongNumberChecker():
    assert ArmstrongNumberChecker(153) == True
    assert ArmstrongNumberChecker(370) == True
    assert ArmstrongNumberChecker(371) == True
    assert ArmstrongNumberChecker(9474) == True
    assert ArmstrongNumberChecker(123) == False
```

Output:

```
PS C:\Users\HP\OneDrive\Documents\AIAC_2026> python -m pytest Lab_08.py
========================================================= test session starts =========================================================
platform win32 -- Python 3.13.5, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\HP\OneDrive\Documents\AIAC_2026
collected 1 item

Lab_08.py .                                                                                                                    [100%]

========================================================== 1 passed in 0.04s ==========================================================
```